C for Pebble Programmers - Emery course syllabus

Version: revised 41-activity self-directed syllabus - Target: Pebble Time 2 (Emery, 200 x 228) - Language: C - SDK: 4.33.1 - Focused study estimate: 52h 35m - Schedule: self-paced with prerequisite gates.

Course purpose

Learn to design, implement, understand, test and independently extend Pebble apps. This is not merely a catalog of SDK functions: the technical capabilities support seven essential competencies, revisited throughout the course. You progress from a time-only watchface to three evolving watchapps and finally to an independently conceived and built app. No university enrollment, attendance, grade, or fixed deadline is implied.

Seven learning outcomes

By the end of the course, demonstrate all seven abilities independently:

UI Design
Measurable outcome: Create legible Emery layouts, navigate between screens, and communicate states and actions clearly.

Idea Generation
Measurable outcome: Develop alternatives, evaluate constraints, and justify an original app concept or representation.

Algorithmic Thinking
Measurable outcome: Trace execution, specify state/loop logic, and validate state transitions and boundary conditions.

C Programming
Measurable outcome: Write and explain C data/control flow, pointers, callback functions, object ownership, and resource safety.

Resource Literacy
Measurable outcome: Interpret SDK signatures and limits, locate sources, and make evidence-based API choices.

Debugging and Testing
Measurable outcome: Reproduce issues, form hypotheses, make minimal fixes, and verify regression and negative tests.

Development Tools and Workflow
Measurable outcome: Build, test, log, use Git, manage integration artifacts, and reproduce results independently.

Each of the 40 preparatory activities has a primary and supporting competency; the capstone is evaluated against all seven. Primary task distribution: 10 C programming; 5 each in the other six competencies. Hours may vary by task; equal task counts do not imply equal time.

Complete outcomes, evidence and capstone rubric - All 41 activities mapped to competencies

What you need

Pebble SDK 4.33.1, the Emery emulator, C build tools, Git, editor, and local source files. Use an actual Pebble Time 2 where sensor calibration, vibration, backlight or physical device behavior must be confirmed; phone communications also need a connected compatible Pebble mobile environment. Do not present an emulator result as physical hardware verification. Official C reference.

Start here: Lesson index - C readings 4A-4H - Pebble API track - Taskwarrior instructions.

Learning method and assessment

For every lesson: (1) Read and plan: study the linked local reading/SDK signatures and express the intended change in your own words. (2) Implement and test: work from the local starter, commit your change, build/run, and test a normal plus a boundary/failure case. (3) Reflect and transfer: explain state and resource ownership, record a problem or trade-off, and name one additional use. Existing activities already specify their deliverables; these steps organize rather than replace them.

Score the activity's primary and supporting competency at 0-3 using the common rubric. Both must reach 2 or 3, and the stated acceptance criteria must pass, before marking the activity complete. Do not conflate unverified hardware behavior with an implemented fallback.

0 - Not demonstrated
Standard: No functional evidence or intelligible explanation

1 - Developing
Standard: Partially demonstrated, with critical understanding/test gaps

2 - Competent
Standard: Correct or appropriately scoped result, independent explanation, normal and boundary tests, and saved evidence

3 - Transferable
Standard: Competent plus independent generalization to a new context with justified trade-offs

Independent-work policy: Do not use AI-generated source, solutions, code explanations, diagrams or debugging fixes to complete exercises. Consult included starters, official SDK documentation, logs, compiler diagnostics and your own notes. Versioned evidence is required. Detailed scoring and readiness gates.

Optional self-check (pilot): After you finish selected activities (7A, 4A in v1), you may run the local assessor tool (assess submit <ID>) to compare your Git-backed evidence and EVIDENCE.md against course criteria and meets/below exemplars. It uses spaCy, not generative AI, does not help during exercises, and does not mark Taskwarrior tasks complete.

Recommended integrated course progression

The sequence interleaves C learning with UI work, ideation, algorithm design, research, debugging and tools. The API material no longer sits as a detached final block. Preserve all original IDs, and use the "Depends on" column and Taskwarrior to work out of order safely.

I Orientation and fundamentals
7 | 4h 45m

II Control flow, code reading and first window
9 | 10h 45m

III Memory, design, events and interactive foundations
14 | 19h 00m

IV Graphics, hardware, validation and integration
10 | 14h 05m

V Independent application capstone
1 | 4h 00m

Phase I - Orientation and fundamentals

Readiness checkpoint: Working time-only watchface, baseline Git workflow and independent diagnostic explanation.

2A Explore Pebble examples
Time 20m | Primary: Idea Generation | Supporting: Resource Literacy
Prerequisites: None

7A Build and run a time-only C watchface
Time 30m | Primary: Development Tools and Workflow | Supporting: C Programming
Prerequisites: None

4A Variables, Data Types, and Operators
Time 120m | Primary: C Programming | Supporting: Debugging and Testing
Prerequisites: 7A

5A Find and document a Pebble API function
Time 20m | Primary: Resource Literacy | Supporting: C Programming
Prerequisites: 4A

1A Change one visual property of a time-only watchface
Time 30m | Primary: UI Design | Supporting: Resource Literacy
Prerequisites: 4A, 5A, 7A

6A Diagnose a simple compiler error
Time 20m | Primary: Debugging and Testing | Supporting: Development Tools and Workflow
Prerequisites: 4A, 7A

7B Create a Git-based modification workflow
Time 45m | Primary: Development Tools and Workflow | Supporting: Debugging and Testing
Prerequisites: 7A, 1A, 6A

Phase II - Control flow, code reading and first window

Readiness checkpoint: Explain control flow, trace callbacks, design an initial UI and articulate a bounded algorithm.

4B Conditional Execution and Decision Making
Time 120m | Primary: C Programming | Supporting: Algorithmic Thinking
Prerequisites: 4A

4C Loops and Iteration
Time 135m | Primary: C Programming | Supporting: Algorithmic Thinking
Prerequisites: 4B

4D Functions and Program Structure
Time 150m | Primary: C Programming | Supporting: Algorithmic Thinking
Prerequisites: 4C

3A Trace the watchface lifecycle
Time 30m | Primary: Algorithmic Thinking | Supporting: C Programming
Prerequisites: 4D, 7A

8A Windows and layers
Time 60m | Primary: UI Design | Supporting: Development Tools and Workflow
Prerequisites: 7A, 3A

1B Add and style a date TextLayer
Time 45m | Primary: UI Design | Supporting: C Programming
Prerequisites: 1A, 4D, 5A

5B Trace an existing feature through source code
Time 45m | Primary: Resource Literacy | Supporting: Algorithmic Thinking
Prerequisites: 3A, 4D, 5A

3B Design a font-cycling algorithm
Time 30m | Primary: Algorithmic Thinking | Supporting: UI Design
Prerequisites: 3A, 5B

2B Write a two-example remix brief
Time 30m | Primary: Idea Generation | Supporting: UI Design
Prerequisites: 2A, 5B

Phase III - Memory, design, events and interactive foundations

Readiness checkpoint: Demonstrate C memory/state, original concept and interactive app behavior with tests.

4E Arrays, Strings, and Bounds
Time 90m | Primary: C Programming | Supporting: Debugging and Testing
Prerequisites: 4D

4F Pointers, Indirection, and Memory Lifetime
Time 105m | Primary: C Programming | Supporting: Debugging and Testing
Prerequisites: 4E

4G Structures, Enums, and Program State
Time 105m | Primary: C Programming | Supporting: Algorithmic Thinking
Prerequisites: 4F

5C Research an unfamiliar Pebble capability
Time 60m | Primary: Resource Literacy | Supporting: C Programming
Prerequisites: 4E, 5B

6B Diagnose a font-selection boundary bug
Time 45m | Primary: Debugging and Testing | Supporting: C Programming
Prerequisites: 4E, 6A, 3B

2C Develop an original Pebble app concept
Time 60m | Primary: Idea Generation | Supporting: Resource Literacy
Prerequisites: 2B, 5C

3C Design a state machine for an app
Time 60m | Primary: Algorithmic Thinking | Supporting: UI Design
Prerequisites: 2C, 3B, 4G

4H Event-Driven Integration and Application Lifecycle
Time 150m | Primary: C Programming | Supporting: Development Tools and Workflow
Prerequisites: 4G, 3C

1C Design a multi-mode Pebble interface
Time 90m | Primary: UI Design | Supporting: Idea Generation
Prerequisites: 1B, 2C, 3C, 4H

8B Menus and scrolling
Time 75m | Primary: UI Design | Supporting: Algorithmic Thinking
Prerequisites: 4E, 4H

8C Button interactions
Time 60m | Primary: Algorithmic Thinking | Supporting: C Programming
Prerequisites: 8A, 4H

8D Custom drawing
Time 75m | Primary: Idea Generation | Supporting: UI Design
Prerequisites: 8B, 4D

8E Graphics paths
Time 75m | Primary: Algorithmic Thinking | Supporting: UI Design
Prerequisites: 8D, 4F

8G Accelerometer
Time 90m | Primary: C Programming | Supporting: Resource Literacy
Prerequisites: 4H, 5C

Phase IV - Graphics, hardware, validation and integration

Readiness checkpoint: Verify the three recurring projects, record limitations and diagnose integration faults.

7C Integrate font browsing with persistent selection
Time 150m | Primary: Development Tools and Workflow | Supporting: Debugging and Testing
Prerequisites: 1C, 4H, 5B, 7B

6C Diagnose a persistent-state integration bug
Time 90m | Primary: Debugging and Testing | Supporting: Development Tools and Workflow
Prerequisites: 4H, 6B, 7C

8F Bitmap manipulation
Time 90m | Primary: C Programming | Supporting: Debugging and Testing
Prerequisites: 8E, 4F

8H Magnetometer / compass
Time 90m | Primary: Resource Literacy | Supporting: C Programming
Prerequisites: 8G

8I Ambient-light limits and backlight
Time 45m | Primary: Resource Literacy | Supporting: UI Design
Prerequisites: 8G, 5C

8J Vibration

Time 45m | Primary: Debugging and Testing | Supporting: UI Design
Prerequisites: 8G, 4H

8K App timers

Time 75m | Primary: Debugging and Testing | Supporting: Algorithmic Thinking
Prerequisites: 8C, 4H

8L Resource files

Time 65m | Primary: Idea Generation | Supporting: Resource Literacy
Prerequisites: 8F, 5C

8M Persistent storage

Time 75m | Primary: Development Tools and Workflow | Supporting: Debugging and Testing
Prerequisites: 8K, 4H

8N Phone communication

Time 120m | Primary: Development Tools and Workflow | Supporting: C Programming
Prerequisites: 8L, 4H, 7B

Phase V - Independent application capstone

Readiness checkpoint: Build and defend an original working app using all seven competencies.

9A Build and demonstrate an original Emery app

Time 240m | Primary: Integrated capstone (all seven competencies) | Supporting: All seven competencies
Prerequisites: 2C, 1C, 6C, 8J, 8M, 8N

Three evolving project strands

Clock Workshop

Continuously revised lesson checkpoints: 7A -> 8A -> 8C -> 8K -> 8M | Learning trajectory: Time-only watchface reference -> two-window watchapp -> button behavior -> AppTimer -> persisted setting and recovery

Field Instrument

Continuously revised lesson checkpoints: 8G -> 8H -> 8I -> 8J | Learning trajectory: Accelerometer -> magnetic heading and calibration -> light API limitations -> intentional vibration and negative tests

Pocket Field Guide

Continuously revised lesson checkpoints: 8B -> 8D -> 8E -> 8F -> 8L -> 8N | Learning trajectory: Menu/scrolling -> custom drawing -> GPath -> writable bitmap -> raw resource -> phone messaging

Each new lesson starts from a completed prior-stage reference checkpoint, not a solution to its current exercise. Changes completed in a previous lesson should remain present in later examples. The 7A time-only watchface remains a watchface; the new interactive projects are watchapps. No dependency is imposed between unrelated project strands unless shared prerequisites truly require it. See the API coverage matrix.

Reading and exercises

4A-4D: Read the local chapters adapting Learning C with Pebble (Jipping, 2016), including the 11 distinct local Chapter 3-6 starter projects. 4E-4H: Read the original C-focused course notes on arrays, pointers, structs and event-driven lifecycles. 8A-8N: Consult each lesson's specific C API reference before editing its checkpoint. Keep coding exercises distinct from the written ideation, UI, tracing, resource-research and debugging exercises; none of these are optional simply because they produce no executable code.

Optional debugging challenges: 8C/debug (button navigation), 8I/debug (bad Light API call), 8L/debug (resource extraction); the original compulsory 6A/6B/6C remain. Optional challenges do not increase the required activity count or estimated total.

Independent demonstration - 9A

The capstone implements the original concept developed in 2C and UI groundwork in 1C. You must deliver a working, original Emery app, integrating at least two cross-strand capabilities (or an equivalent justified integration), and defend implementation decisions with source references, normal/failure tests, Git history and a demo. Score each of the seven competencies 0-3; every competency must reach 2+. Detailed capstone directions.

Course management and validation status

Use python3 taskwarrior/import_course.py to synchronize activities, prerequisites and learning objectives, preserving existing completed tasks and personal annotations. To inspect: task project:pebble.course list, task project:pebble.course unblocked, and task <ID> info. Always work in a personal branch/copy, with a known-good baseline.

Source/status transparency: All local lesson/project paths and Emery target manifests should be checked, but C code and phone integrations have not been fully compiled or physically validated in this editing environment. Do not treat a passing manifest audit as a passing SDK build. Maintain separate statuses: source supplied, build passed, emulator passed, real-device passed. Hardware-only validation can remain pending while continuing other independently unlocked tasks.

References

Jipping, M. (2016). Learning C with Pebble. GitBook. Local C course reading index.

Pebble Developers. (n.d.). C SDK documentation. https://developer.repebble.com/docs/c/

Reference links

Complete lessons : https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons

Seven-competency rubric : https://github.com/Darachnid/c-for-pebble-programmers-course/blob/main/curriculum/course-design/learning-outcomes-and-assessment.md

Integrated sequence : https://github.com/Darachnid/c-for-pebble-programmers-course/blob/main/curriculum/course-design/integrated-sequence.md

Course evidence assessor (pilot) : https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/assessor

Official Pebble C SDK : https://developer.repebble.com/docs/c/